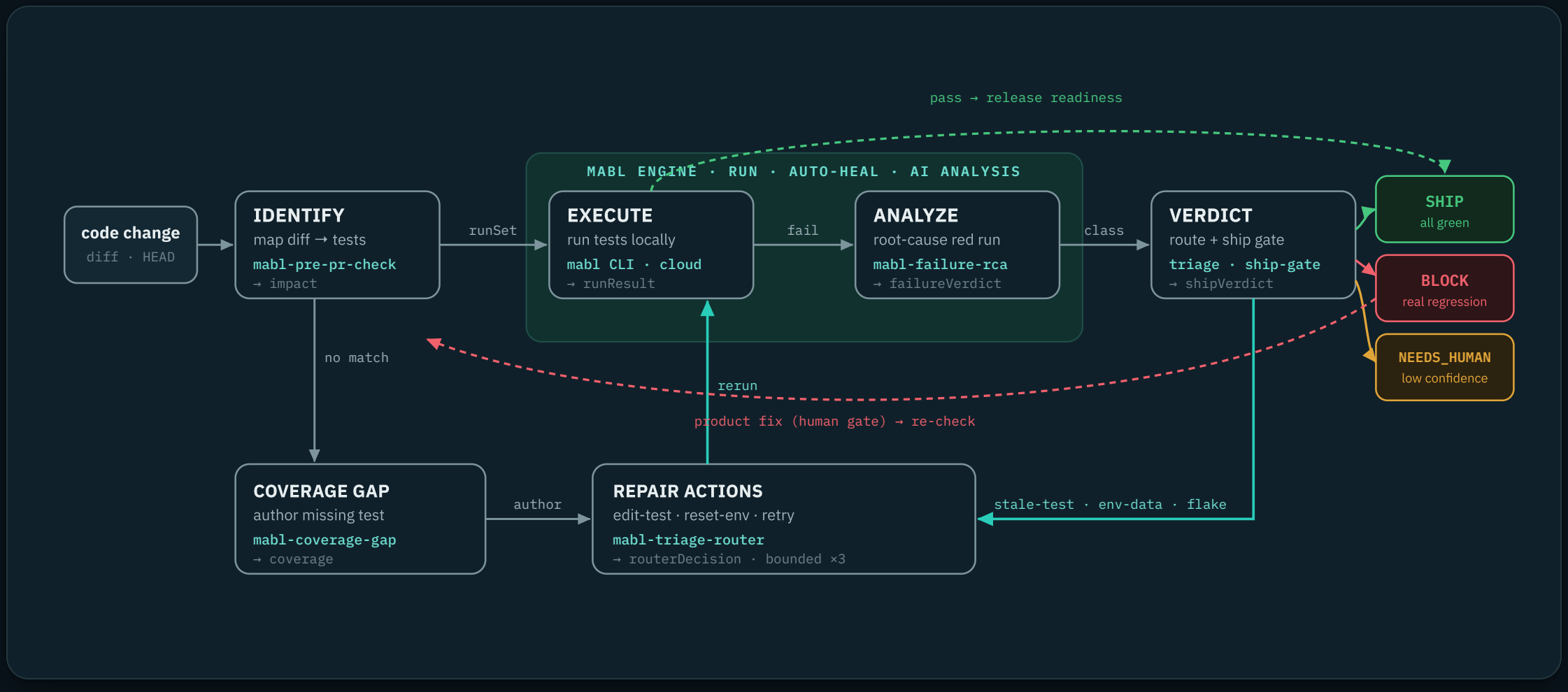
AGENTIC DEVELOPMENT · BANKING-APP

How mabl closes the agentic dev loop

An AI coding agent writes the change; mabl decides whether it's safe. Six repo skills turn a diff into a verdict — **identify** the tests a change touches, **author or update** coverage, **execute** locally, **analyze** every red run against the source, then **act**: loop back to fix-and-rerun, or block the build on a real regression.

`6` skills · runtime **mabl CLI + MCP** · handoff **JSON loop contracts** · bound **3 repair cycles** · gates **code-fix · ship**



The spine runs left to right; the **teal loop** is the self-driving part — a red run is root-caused, and a **stale test, env/data, or flake** auto-repairs and reruns without a human, bounded to three cycles before it escalates. Only two things leave the loop: a clean pass earns a **SHIP** recommendation, and a confirmed **product regression BLOCKS** the build — its code fix is always human-gated (dashed), never auto-applied.

THE SIX SKILLS

Each skill does one job and hands off a machine-readable JSON contract, so the next skill — or a headless runner — branches on it deterministically.

CONDUCTOR

feature-dev

Plan → build → test → ship. Writes the Kiro spec + Jira epic, browser-verifies the build, then composes the five skills below.

`feature idea` → `PR-ready`

Q1 · Q2 — IDENTIFY + EXECUTE

mabl-pre-pr-check

Maps the diff to the mabl tests that exercise the changed flows and runs the best matches locally for pass/fail in minutes.

`diff` → `impact` `runResult`

Q5 — GAP

mabl-coverage-gap

Finds user-facing flows the change touches that no test exercises, rates them, and recommends authoring to close the gap.

`changed flows` → `coverage`

Q3 · Q4 — ANALYZE

mabl-failure-rca

Root-causes a red run against the source using DOM, HAR, console and screenshots — and classifies it: product, stale-test, env-data, or flake.

`red run` → `failureVerdict`

ROUTE

mabl-triage-router

The decision brain. Turns a class into the next action, enforces the 3-cycle bound, and gates product fixes so they never auto-apply.

`failureVerdict` → `routerDecision`

Q6 — SHIP GATE

ship-gate

Checks release readiness against the run signal and policy, returning SHIP / BLOCK / NEEDS_HUMAN. Recommends only — the human clicks merge.

`runResult set` → `shipVerdict`

HOW THE VERDICT ACTS

The failure class decides the next move. Three classes auto-repair and rerun; only a real product regression stops the loop and reaches for a human.

Failure routing

mabl-triage-router · branches on `failureVerdict.class`

CLASS	ACTION	APPLY	NEXT
product	propose-code-fix	human gate	fix code → re-check
stale-test	edit-test	auto	update & rerun
env-data	reset-env	auto	rerun
mabl-flake	retry	auto	rerun
iteration > 3	escalate	human	stop loop, notify

Ship decision

ship-gate · `shipVerdict.decision`

SHIP Affected tests pass, no critical coverage gap. PR gate opens — human merges.

BLOCK Confirmed product regression. Open a bug; the fix loops back through the pipeline.

NEEDS_HUMAN Low-confidence RCA or a critical gap with no test. Escalate for a call.

Sourced from `.kiro/skills/` + `docs/mabl/loop-contracts.md`

plan → build → test → ship